PEMROGRAMAN SISI SERVER

Architectural Patterns & System Design

Suprayogi,M.Kom

Peta Pembelajaran Minggu 1–16

Minggu 2 berada pada fase desain arsitektur sebelum Docker dan Django.

M1
Backend Fundamentals

M2
Architectural Patterns

M3
Docker Containerization

M4
Django & ORM

M5
Database Optimization

M6
REST API

M7
Authentication & Authorization

M8
UTS

M9
NoSQL & Redis

M10
RabbitMQ & Celery

M11
Testing Strategies

M12
CI/CD & Deployment

M13
Integration & Performance

M14
Security & Production

M15
Finalization & Presentation

M16
UAS

Posisi Minggu 2

Dari menjalankan backend sederhana menuju memahami struktur dan batasan module.

Minggu 1

Backend fundamentals, HTTP, Python, venv, Git/GitHub, dan server sederhana.

Minggu 2

Architectural patterns, modularisasi, diagram arsitektur, ERD awal, dan draft resource API.

Minggu 3–4

Docker kemudian Django & ORM. Implementasi database dan framework dilakukan setelah desain dasar.

Batasan Minggu 2

- Belum menggunakan Django, ORM, database, atau Docker.
- Server Python sederhana dipakai sebagai media belajar arsitektur dan modularitas.

Capaian Pembelajaran Minggu 2

- Menjelaskan monolithic, microservices, event-driven, dan serverless secara sederhana.
- Membandingkan modular monolith dan microservices berdasarkan kebutuhan dan trade-off.
- Menjalankan kembali server Python dan menambah contoh endpoint domain Simple LMS.
- Melakukan refactoring dari satu file menjadi beberapa module berdasarkan tanggung jawab.
- Menggunakan Git/GitHub sebagai workflow bertahap.
- Membuat diagram arsitektur dan ERD awal sebagai rancangan, bukan implementasi database.

Fokus utama: konsep → demo dosen → guided coding → latihan desain → diskusi kelas

Alur Kegiatan Kelas Minggu 2

1. Review M1
2. Konsep arsitektur
3. Guided Coding: endpoint
4. Refactoring modules
5. Git workflow
6. Diagram + ERD
7. Diskusi & penutup

Empat Architectural Patterns

Gunakan sebagai model berpikir; bukan daftar teknologi yang harus diimplementasikan minggu ini.

Monolithic

Satu aplikasi/deployment unit. Sederhana untuk memulai, tetapi perlu struktur internal yang jelas.

Microservices

Layanan dipisah berdasarkan capability dan dapat dideploy independen; kompleksitas operasional meningkat.

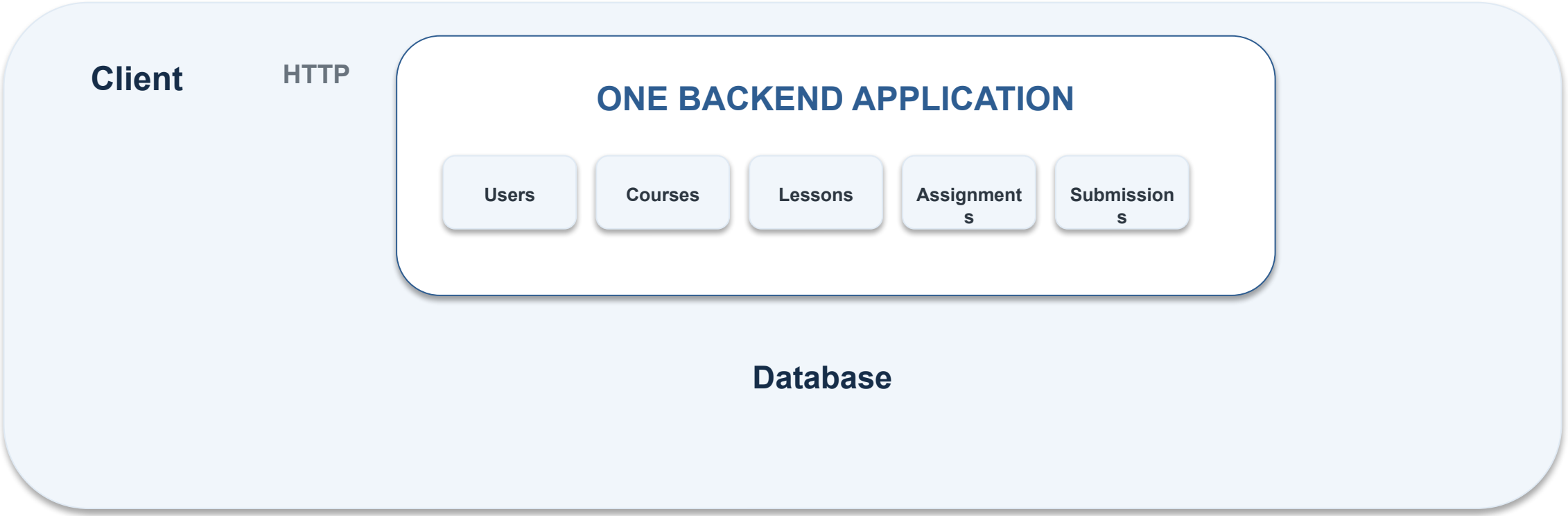
Event-Driven

Komponen berkomunikasi melalui event dan message broker; cocok untuk proses asynchronous.

Serverless

Fungsi dijalankan oleh platform cloud tanpa mengelola server secara langsung; dibahas konseptual.

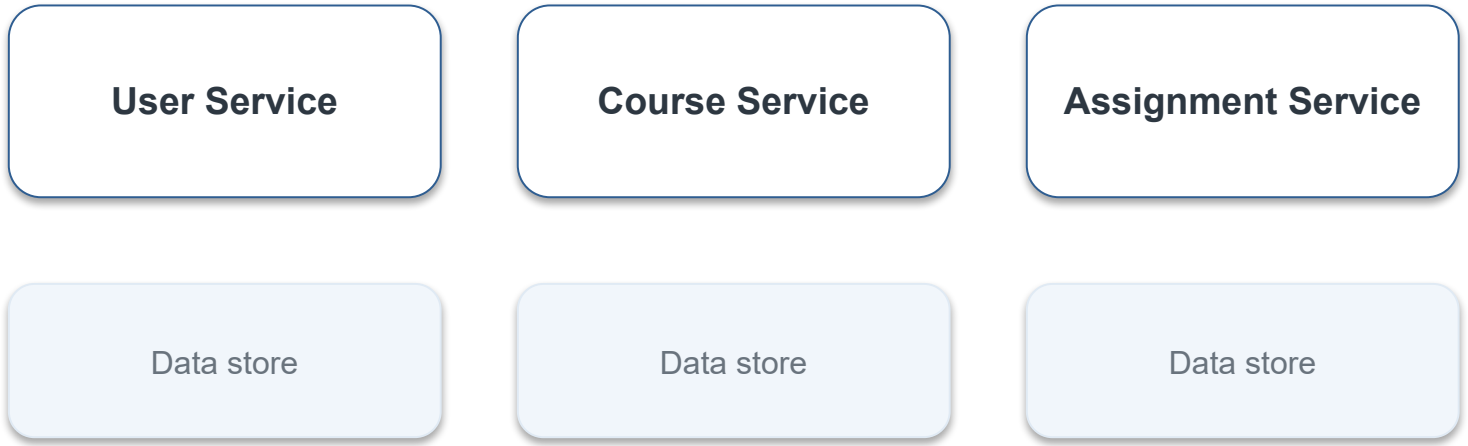
Monolithic Architecture



- Satu aplikasi dan satu deployment unit.
- Struktur internal dapat tetap modular.

- Masalah utama muncul ketika batas module buruk (tdk ada pemisahan tugas yang jelas antar fungsi /fitur) dan coupling meningkat(tingkat ketergantungan antar modul).

Microservices Architecture



- Deployment dapat dipisahkan per service.
- Komunikasi network, observability, security, dan data consistency menjadi perhatian tambahan.
- Bukan berarti semua aplikasi harus dipecah menjadi microservices.

Monolith vs Microservices: Trade-off

Aspek	Monolith	Microservices
Deployment	Satu unit	Dapat dipisah per service
Komunikasi	In-process	Network/API/message
Debugging	Relatif terpusat	Lintas service lebih kompleks
Data	Lebih mudah dikelola bersama	Konsistensi lintas service perlu strategi
Skala	Sering scale seluruh aplikasi	Dapat scale service tertentu
Operasional	Lebih sederhana di awal	Membutuhkan maturity operasional lebih tinggi

Inti pembelajaran: bandingkan kebutuhan dan trade-off, bukan mencari satu pola yang selalu paling baik.

Event-Driven Architecture (EDA)



Pola arsitektur perangkat lunak yang menggunakan events untuk menghubungkan dan mengomunikasikan komponen-komponen sistem yang terpisah .

Komponen Utama:
- Producer: Komponen yang membuat dan menerbitkan (publish) data event.
- Event / Broker: Perantara yang menerima, menyimpan, dan menyalurkan event. misl Kafka.RabbitMQ.
- Consumer : Komponen yang menerima dan memproses data event tersebut.

- Relevan untuk proses asynchronous dan integrasi antarkomponen.
- Trade-off: tracing alur dan eventual consistency dapat menjadi lebih kompleks.

Contoh implementasi Event-Driven Architecture (EDA)

Sistem aplikasi ride-hailing (Gojek /Grab) saat Anda memesan makanan atau kendaraan. Berikut adalah bagaimana EDA bekerja di balik layar aplikasi tersebut:

- Pemicu Event: Anda menekan tombol "Pesan Sekarang". Tindakan ini membuat sebuah event bernama OrderPlaced.
- Peran Broker: Aplikasi mengirim event tersebut ke sebuah Event Broker (seperti Apache Kafka).
- Aksi Para Consumer: Berbagai layanan (consumers) langsung merespons event OrderPlaced secara bersamaan tanpa saling menunggu:
  - Layanan Driver: Mencari driver terdekat untuk mengambil pesanan.
  - Layanan Restoran: Mengirim notifikasi ke resto agar makanan mulai dimasak.
  - Layanan Promo: Memeriksa dan memotong kupon diskon yang Anda pakai.
  - Layanan Notifikasi: Mengirim pesan ke ponsel Anda bahwa pesanan sedang diproses.

Serverless: Konsep Dasar

Trigger

HTTP, upload, event, atau scheduler

Function

Fungsi berjalan saat dipanggil

Platform

Provider mengelola runtime dan infrastruktur (AWS Lambda, GoogleCloud, Functions, Azure Functions)

Output

Respons atau efek samping ke layanan lain

sebuah model komputasi cloud di mana developer dapat menjalankan kode aplikasi tanpa harus mengelola server secara mandiri.

- Pertimbangkan cold start, batas waktu eksekusi, vendor lock-in, dan pola biaya sesuai workload.

contoh aplikasi Serverless

Aplikasi Pemroses Gambar & Video (Contoh: Fitur Profil Instagram/YouTube)

- Skenario: Pengguna mengunggah foto profil baru. Foto tersebut harus otomatis dikompres, diubah ukurannya menjadi beberapa versi (thumbnail, ukuran sedang, ukuran penuh), dan disimpan ke penyimpanan cloud.
- Mengapa Serverless Cocok?
  - Proses ini dipicu oleh event (user mengunggah gambar).
  - Fungsi serverless hanya berjalan selama beberapa detik saat gambar diunggah, lalu mati. Anda tidak perlu membayar server yang menyala 24 jam hanya untuk menunggu pengguna mengganti foto profil.

Bagaimana Memilih Arsitektur?

Skala sistem	Project latihan di kelas berbeda dengan platform nasional(yg hrs menangani jutaan user).
Ukuran tim	Tim kecil biasanya membutuhkan operasional yang sederhana(Monolith).
Deployment independen	Jika tidak dibutuhkan, pemisahan service mungkin belum diperlukan.
Proses asynchronous	Event-driven dapat relevan untuk pekerjaan latar/notifikasi .
Kompleksitas	Setiap pola membawa konsekuensi pada debugging dan operasional.

Prinsip: arsitektur yang baik adalah arsitektur yang sesuai kebutuhan.

Pilihan Arsitektur untuk project Simple LMS

BASELINE PEMBELAJARAN

Modular Monolith

- Satu backend application / satu deployment unit.
- Module boundary: Users, Courses, Enrollments, Lessons, Assignments, Submissions.
- Service boundaries tetap dirancang agar konsep evolusi arsitektur dipahami.

Mengapa tidak langsung microservices?

- Lingkup proyek perkuliahan masih terbatas.
- Operasional dan debugging perlu tetap sederhana.
- Tujuannya memahami boundary sebelum menambah kompleksitas deployment.

Guided Coding 1 — Tambah Endpoint Simple LMS

```
GET  /
GET /health
GET /courses
GET /students
GET /assignments
```

```
if self.path == "/courses":
   self.send_json({"courses": [
      {"id": 1, "name": "Pemrograman Sisi Server"},
      {"id": 2, "name": "Basis Data"}
   ]})
```

- Tujuan: satu backend melayani beberapa resource.
- Uji melalui browser / HTTP tool.
- Perhatikan request → endpoint → handler → response.

Membaca Alur Request → Response

Client

GET /courses

Router / Handler

Cocokkan path dan method

Domain Module

Ambil data courses

Response

JSON + status code

Latihan tampilkan minimal 4 response JSON dan satu response 404.

Mengapa Kode Perlu Dimodularisasi?

- server.py akan semakin panjang ketika domain bertambah.
- Data dan logic berbagai domain mudah bercampur.
- Tanggung jawab tiap bagian menjadi kurang jelas.
- Perubahan satu fitur lebih mudah memengaruhi bagian lain.
- Kolaborasi tim lebih sulit ketika semua bekerja pada file yang sama.

Masalah tersebut menjadi jembatan menuju module boundary dan modular monolith.

Guided Coding 2 — Refactoring menjadi Modules

```
simple-lms-backend/
├── server.py
├── modules/
│   ├── __init__.py
│   ├── courses.py
│   ├── students.py
│   └── assignments.py
├── .gitignore
└── README.md
```

- Satu module = satu tanggung jawab domain.
- server.py menangani HTTP; module menangani data/logic domain sederhana.
- Aplikasi tetap satu server dan satu proses.

```
from modules.courses import get_courses

elif self.path == "/courses":
    self.send_json({"courses": get_courses()})
```

Contoh Module: Courses

```
# modules/courses.py
courses = [
   {"id": 1, "name": "Pemrograman Sisi Server"},
   {"id": 2, "name": "Basis Data"}
]

def get_courses():
   return courses
```

Tanggung jawab

Menyediakan data/operasi domain Courses untuk dipanggil oleh server.

Bukan microservices

Tidak ada deployment terpisah, tidak ada komunikasi network antarmodule.

Modular Monolith: Apa yang Berubah?

SEBELUM

```
server.py

HTTP + Users + Courses
+ Students + Assignments
+ data + logic
```

SESUDAH

```
server.py
modules/
  courses.py
  students.py
  assignments.py
```

Deployment tetap satu; organisasi kode menjadi lebih jelas.

Git & GitHub sebagai Workflow Berkelanjutan

```
# setelah endpoint
git add .
git commit -m "feat: add LMS endpoints"

# setelah refactoring
git add .
git commit -m "refactor: organize backend into modules"

# dokumentasi
git add .
git commit -m "docs: add architecture notes"
git push origin main
```

- Commit setelah perubahan logis, bukan satu commit besar di akhir.
- GitHub menjadi jejak evolusi pekerjaan.
- Latihan di kelas: cek git log dan diskusikan perubahan tiap commit.

Checkpoint kelas: repository menunjukkan beberapa tahap perubahan.

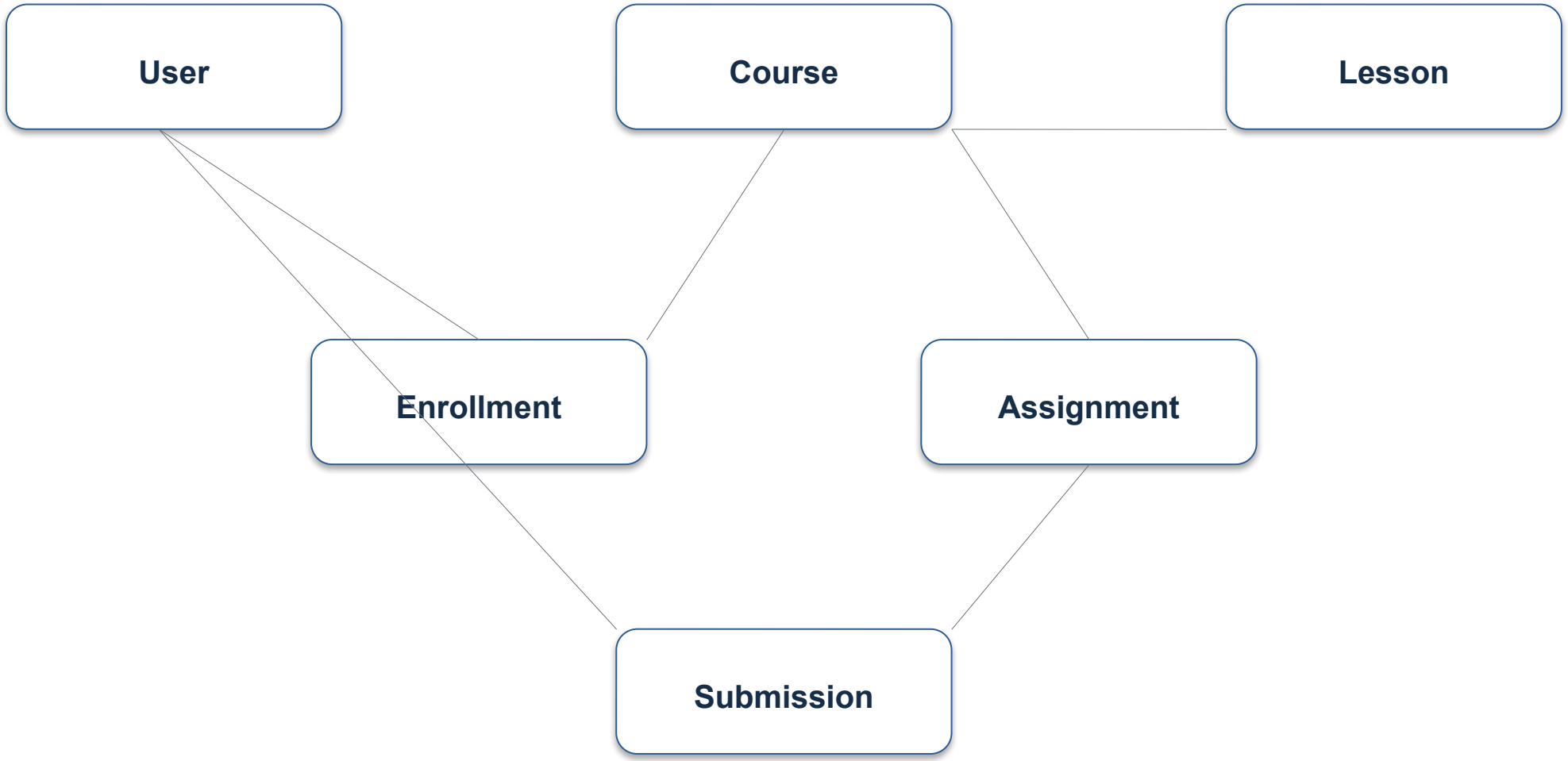
Latihan Desain — Dari Kode ke Diagram



Latihan gambarkan komponen dan hubungan berdasarkan kode yang baru saja dibuat.

ERD Awal Simple LMS

Rancangan awal saja; implementasi database dan ORM baru masuk Minggu 4.



Contoh field: User(id, name, email, role) • Course(id, code, title, lecturer) • Enrollment(user_id, course_id)

Draft API Awal: Resource, Bukan Tombol UI

Method	Endpoint	Tujuan
GET	**/courses**	Daftar course
GET	**/courses/{id}**	Detail course
GET	**/lessons**	Daftar lesson
GET	**/assignments**	Daftar assignment
GET	**/enrollments**	Daftar enrollment

Prinsip: endpoint merepresentasikan resource/domain, bukan nama tombol seperti /klik-tombol-course.

Architecture Decision Record (ADR) Sederhana

Context

Simple LMS masih berada pada tahap awal; tim perlu struktur yang jelas tanpa menambah kompleksitas operasional.

Decision

Gunakan modular monolith sebagai baseline pembelajaran.

Trade-off

Lebih sederhana untuk dijalankan; belum memberi isolasi deployment seperti microservices.

Future

Service boundary tetap terdokumentasi sehingga evolusi arsitektur dapat dibahas kemudian.

Latihan diskusi: apa bukti kebutuhan yang harus muncul sebelum kita memecah service?

buktinya harus berupa data nyata terkait bottleneck operasional (CPU, RAM, trafik), hambatan deployment, atau friksi koordinasi tim.

Latihan

- buat lagi server Python di Minggu 2 (lihat server.py di modul1).
- Tambahkan endpoint /courses, /students, /assignments.
- Uji response JSON dan satu 404.
- Pindahkan data/logic sederhana ke folder modules/.
- Buat commit setelah endpoint dan setelah refactoring.
- Gambarkan arsitektur sederhana dan ERD awal.

Checkpoint Pemahaman

Checkpoint 1

Server dan repository siap.

Checkpoint 2

Endpoint berjalan + mahasiswa dapat menjelaskan alur request/response.

Checkpoint 3

Refactoring menjadi modules berhasil dan Git menunjukkan evolusi.

Checkpoint adalah pemeriksaan belajar di kelas

Konteks Capstone: System Architecture

Simple LMS akan berkembang bertahap pada minggu-minggu berikutnya.

Users	**Courses**	**Enrollments**
Lessons	**Assignments**	**Submissions**

- Hari ini kita mengenali boundary dan hubungan antarbagian.
- Belum ada implementasi database, ORM, Docker, atau framework Django.
- Rancangan akan menjadi konteks ketika implementasi dimulai.

Penutup Minggu 2

Hari ini kita belajar bahwa arsitektur bukan sekadar memilih teknologi.

- Kita membandingkan pola arsitektur dan trade-off-nya.
- Kita melihat modular monolith melalui refactoring kode nyata.
- Kita menghubungkan kode dengan diagram arsitektur, ERD awal, dan draft API.
- Tidak ada penugasan pada Minggu 2; fokusnya memahami konsep dan berlatih bersama di kelas.